

Lightweight Martian Terrain Classification for Rover Safety: A Comparative Study of CNN, MobileNetV2, and SNN

Venkata Naga Sai Bhavan Koppolu and Daniel Lewis

MAE 494/598: Introduction to Autonomous Vehicles

April 28, 2026

C contents

Chapter 1: Introduction.....	5
1.1 Background.....	5
1.2 Problem Statement	5
1.3 Objectives	6
1.4 Research Questions	6
Chapter 2: Literature Review.....	7
2.1 Terrain Classification for Autonomous Navigations.....	7
2.2 Lightweight Deep Learning Models	7
2.3 Spiking Neural Networks (SNNs).....	7
2.4 Challenges in Mars-Based AI Systems	8
2.5 Research Gap.....	8
Chapter 3: System Requirements	9
3.1 Hardware Requirements.....	9
3.2 Software Requirements	9
Chapter 4: Datasets.....	10
4.1 Dataset A	10
4.2 Dataset B	10
Chapter 5: Method.....	11
5.1 Preprocessing Pipelines	11
5.1.1 Minimal Preprocessing (A1 / B1)	11
5.1.2 Augmented Preprocessing (A2 / B2)	11
5.2 Model Architectures	11
5.2.1 Tiny CNN	11
5.2.2 MobileNetV2	12
5.2.3 Spiking Neural Network (SNN)	12
5.3 Training Configuration	12

5.4	Evaluation Metrics.....	13
5.5	Energy Measurement Methods.....	13
5.5.1	MAC-Based Theoretical Estimation.....	13
5.5.2	CodeCarbon Real Measurement.....	13
Chapter 6:	Results	14
6.1	Accuracy Results	14
6.1.1	Confusion Matrix Analysis	15
6.2	Model Size and Inference Speed	16
6.3	Energy Consumption	17
6.3.1	MAC-Based Theoretical Estimation.....	17
6.3.2	CodeCarbon Real Measurement.....	18
Chapter 7:	Discussion	19
7.1	Impact of Data Augmentation	19
7.2	TinyCNN vs MobileNetV2	19
7.3	SNN Performance and Hardware Dependency.....	20
7.4	Energy Consumption Consideration	20
Chapter 8:	Conclusion.....	21
8.1	Conclusions	21
8.2	Limitations.....	22
8.3	Future Work	22
References		23
Appendix.....		24

T ables & Figures

Table 1:	Hardware Requirements	9
Table 2:	Software Requirements	9
Table 3:	Dataset A Composition	10
Table 4:	Dataset B Composition	10
Table 5:	Accuracy results for all model-experiment combinations.....	14
Table 6:	Model size and inference speed metrics	16
Table 7:	MAC-based theoretical energy per inference	17
Table 8:	CodeCarbon real measured energy per inference	18
Fig 1:	Accuracy.....	14
Fig 2:	Confusion Matrix for Experiment A1	14
Fig 3:	Confusion Matrix for Experiment A2	14
Fig 4:	Confusion Matrix for Experiment B1	14
Fig 5:	Confusion Matrix for Experiment B2	14
Fig 6:	Model Size.....	16
Fig 7:	Inference Time.....	16
Fig 8:	MACs per Inference.....	17
Fig 9:	Theoretical energy per inference comparison on T4	17
Fig 10:	Theoretical energy per inference comparison on Loihi.....	17
Fig 11:	Real energy per inference	18

Chapter 1: Introduction

1.1 Background

The exploration of Mars has been one of humanity's most ambitious scientific endeavours. Since Sojourner's 18-metre journey in 1997, rover missions have grown exponentially in capability and scientific scope. Spirit, Opportunity, Curiosity, and Perseverance have collectively travelled tens of kilometres across the Martian surface, gathering geological and atmospheric data critical to understanding the planet's history and habitability. China's Zhurong rover, deployed in 2021, further demonstrates that Mars exploration is a global effort with increasing momentum.

As these missions grow in ambition, with future goals including sample return and human precursor missions, the demands placed on onboard autonomy increase correspondingly. Rovers cannot rely on real-time human guidance: the one-way communication delay between Earth and Mars ranges from 4 to 24 minutes depending on orbital position, making a round-trip command cycle potentially 48 minutes long. By the time a human operator identifies a hazard and issues a corrective command, the rover may already be damaged or immobilised.

Terrain classification, the automated identification of surface material types from onboard imagery, is a foundational capability for autonomous rover navigation. Correctly classifying terrain as safe soil or bedrock versus hazardous sand or impassable rock directly determines whether a rover can safely traverse a path. The 2009 loss of the Spirit rover, which became permanently stuck in soft soil, illustrates the catastrophic consequences of terrain misclassification.

1.2 Problem Statement

Despite growing interest in applying deep learning to Mars terrain classification, most existing approaches rely on heavyweight models such as ResNet-101 or Xception that are unsuitable for deployment on rover-class hardware. Curiosity's onboard computer runs at 200 MHz with 256 MB of RAM; Zhurong's processor operates at just 25 MHz with 20 MB of RAM. A single inference pass through ResNet-101 requires approximately 60.9 million multiply-accumulate operations (MACs) and takes an estimated 170 milliseconds on Zhurong's processor which is far too slow for real-time navigation.

Furthermore, energy consumption is a critical constraint. Rovers operate on fixed power budgets shared across locomotion, heating, scientific instruments, and communication systems. The inference energy of a terrain classification model must be minimised without sacrificing the accuracy required for safe navigation. This creates a fundamental tension: the models most capable of accurate terrain classification are precisely those that cannot be deployed on the hardware available.

This project addresses this gap by systematically comparing three lightweight neural network architectures: a custom Tiny CNN, a fine-tuned MobileNetV2, and a Spiking Neural Network, for Mars terrain classification under realistic compute and energy constraints.

1.3 Objectives

To address this problem, the project aims to:

- 1) Evaluate and compare multiple model architectures, including:
 - A lightweight Convolutional Neural Network (Tiny CNN)
 - A mobile-optimized architecture (MobileNet)
 - A biologically inspired model (Spiking Neural Network, SNN)
- 2) Analyze model performance across different datasets and preprocessing strategies, specifically:
 - A smaller subset dataset (1000Samples)
 - A larger, real-world dataset (AI4Mars)
 - Minimal preprocessing vs augmented preprocessing
- 3) Measure key performance metrics, including:
 - Classification accuracy
 - Model size (number of parameters)
 - Inference speed (latency)
 - Computational cost (MACs)
 - Energy consumption (theoretical and measured)
- 4) Investigate trade-offs between performance and efficiency, particularly in relation to:
 - Model complexity
 - Hardware constraints
 - Task difficulty
- 5) Assess the practicality of deploying different model types in real-world rover scenarios, including the impact of hardware on model performance.

1.4 Research Questions

- Which lightweight neural network architecture achieves the best balance of classification accuracy and computational efficiency for Martian terrain classification under rover-class hardware constraints?
- Does the theoretical energy advantage of the SNN predicted by MAC-based estimates materialise in practice on conventional GPU hardware, or is it strictly hardware-dependent as suggested by Kucik & Meoni (2021)?
- Is energy consumption a significant constraint in terrain classification tasks compared to other performance metrics?
- How does dataset complexity influence model performance and generalization?
- How does data augmentation affect model performance under limited training conditions?

Chapter 2: Literature Review

2.1 Terrain Classification for Autonomous Navigation

Terrain classification is a critical component of autonomous robotic navigation, particularly in planetary exploration where real-time human intervention is limited. Vision-based approaches are widely used because they allow the system to classify terrain before traversal, improving navigation safety and planning.

Prior research shows that deep learning models, especially Convolutional Neural Networks (CNNs), achieve strong performance in terrain classification tasks by learning discriminative visual features. However, these approaches often require high computational resources, making them difficult to deploy on embedded robotic systems with strict power and hardware constraints (Ibrahim & Al-Jamali, 2024).

2.2 Lightweight Deep Learning Models

To address these limitations, recent work has focused on designing lightweight architectures that reduce computational cost while maintaining performance. Models such as MobileNet use depthwise separable convolutions to significantly reduce parameters and FLOPs.

For example, Mobile-DeepRFB demonstrates that lightweight architectures can reduce parameter count by up to 94% while improving accuracy on Mars terrain datasets (Feng et al., 2025). These approaches highlight the importance of balancing efficiency and feature extraction capability for deployment on low-power platforms such as Mars rovers.

Despite these improvements, lightweight models still face trade-offs between model size and robustness in complex environments.

2.3 Spiking Neural Networks (SNNs)

Spiking Neural Networks (SNNs) provide a biologically inspired alternative to traditional deep learning models. Unlike standard neural networks, SNNs process information through discrete spikes over time, enabling event-driven computation and potential energy savings.

Research indicates that SNNs can achieve improved energy efficiency, particularly when deployed on neuromorphic hardware such as Loihi or SpiNNaker (Kucik & Meoni, 2021). However, their effectiveness depends on factors such as input representation and hardware support.

In practice, SNNs often exhibit lower accuracy compared to conventional CNNs and require more complex training procedures, limiting their widespread adoption.

2.4 Challenges in Mars-Based AI Systems

Deploying AI systems for Mars exploration introduces unique constraints, including limited computational resources, strict energy budgets, and highly variable terrain conditions.

Recent studies emphasize that many high-performance deep learning models are unsuitable for onboard deployment due to their computational complexity. Lightweight models tailored for Mars environments aim to address this by optimizing both efficiency and accuracy while maintaining real-time performance (Lin et al., 2025).

These challenges highlight the need for models that balance accuracy, efficiency, and deployability.

2.5 Research Gap

Although prior work has explored high-accuracy CNNs, lightweight architectures, and energy-efficient SNNs, there is limited research that directly compares these approaches under consistent experimental conditions.

Specifically, existing studies do not comprehensively evaluate:

- Accuracy vs computational complexity
- Energy consumption vs performance
- Model behavior across varying dataset conditions

Chapter 3: System Requirements

3.1 Hardware Requirements

ID	Requirement Description	Comments
1	Internet-connected computer/laptop	Necessary to access Google services, which runs on cloud servers.
2	8 GB RAM	Ensures smooth operation of local tasks and browser stability.
3	Stable high-speed internet connection	Critical for uninterrupted access to Google services and data transfer.

Table 1: Hardware Requirements

3.2 Software Requirements

ID	Requirement Description	Version	Comments
1	Opera GX browser	107.0.5045.60	Required for optimal use of the Google services.
2	Google account	N/A	Required to log in and use Google services.
3	Google Drive	N/A	Provides essential cloud storage for storing the dataset.
4	Google Colab	N/A	Provides the cloud-based Jupyter notebook environment.
5	PyTorch	2.11.0	Necessary library for model training and evaluation.
6	snnTorch	0.9.4	Necessary library for SNN model training and evaluation.
7	CodeCarbon	3.2.6	Necessary library for energy real-time measurement
8	Hugging Face	N/A	Allows access to dataset for training and evaluation of the models.
9	Jupyter Notebook	6.5.7	Useful for local development before moving to Google Colab.
10	GitHub	3.17	Useful for managing code, collaboration, and version control.

Table 2: Software Requirements

Chapter 4: Datasets

Two publicly available datasets hosted on Hugging Face were used for this study. Both were formatted for three-class classification (Soil, Bedrock, Sand) and split 80/20 for training and testing with a fixed random seed of 42 to ensure reproducibility. All images were resized to 64×64 pixels and converted to single-channel grayscale as input to all three models.

4.1 Dataset A: 1000Samples

The 1000Samples dataset (Ankhitan/1000Samples on Hugging Face) is a preprocessed subset of the AI4MARS dataset containing 932 labeled images across three terrain classes. Labels are provided directly at the image level, making it suitable for classification without additional mask processing. The dataset exhibits a relatively balanced class distribution as shown in Table 1.

Class	Image Count	Proportion
Soil	260	27.9%
Bedrock	287	30.8%
Sand	385	41.3%

Table 3: Dataset A Composition

4.2 Dataset B: AI4MARS

The AI4MARS dataset (hassanjbara/AI4MARS on Hugging Face) is NASA's full-scale Mars terrain dataset derived from images captured by the Curiosity, Opportunity, and Spirit rover missions. The original dataset contains 16,064 labelled images with pixel-level semantic segmentation masks annotated by citizen scientists and validated by NASA mission planners.

For this project, pixel-level masks were converted to image-level class labels using majority voting, the class with the highest pixel count in the mask was assigned as the image label. Images with fewer than 100 valid labelled pixels were excluded. Due to Google Colab memory constraints, a maximum of 5,000 samples were loaded from the training split. The resulting class distribution reflects the natural imbalance present in the AI4MARS dataset, with Bedrock being the dominant class at approximately 46% of labels. Table 2 summarises the dataset characteristics.

Class	Image Count	Proportion
Soil	1915	38.3%
Bedrock	2305	46.1%
Sand	780	15.6%

Table 4: Dataset B Composition

Chapter 5: Method

5.1 Preprocessing Pipelines

Two preprocessing pipelines were applied to each dataset, resulting in four dataset-preprocessing combinations (A1, A2, B1, B2) and a total of twelve model-experiment pairs.

5.1.1 Minimal Preprocessing (A1 / B1)

The minimal pipeline establishes a clean baseline with no training augmentation. It applies Resize to 64×64 pixels, conversion to single-channel grayscale, conversion to tensor, and normalisation with mean 0.5 and standard deviation 0.5. This pipeline is intended to measure intrinsic model capability without the confounding effect of augmentation.

5.1.2 Augmented Preprocessing (A2 / B2)

The augmented pipeline adds stochastic transformations to the minimal pipeline to simulate the variability encountered in real Mars terrain imagery. In addition to resizing and normalisation, it applies Random Horizontal Flip ($p=0.5$), Random Rotation of ± 15 degrees (± 20 degrees for AI4Mars), and Colour Jitter with brightness and contrast variation of ± 0.3 (± 0.4 and ± 0.5 respectively for AI4Mars). The augmented pipeline is intended to improve model generalisation by exposing training samples to a wider distribution of image conditions.

5.2 Model Architectures

Three architectures were implemented in PyTorch and evaluated under identical conditions. All models accept single-channel 64×64 grayscale images and output logits for three terrain classes.

5.2.1 Tiny CNN

The Tiny CNN is a custom three-layer convolutional network serving as the lightweight baseline. It consists of three convolutional blocks, each comprising a convolutional layer, Batch Normalisation, ReLU activation, and Max Pooling. Channel progression is $1 \rightarrow 16 \rightarrow 32 \rightarrow 64$. The convolutional blocks are followed by a fully connected head with 4,096 input features (64 channels $\times$ 8×8 spatial), a 256-unit hidden layer with ReLU activation and 30% Dropout, and a final linear layer projecting to three output classes.

5.2.2 MobileNetV2

The MobileNetV2 model uses the pretrained ImageNet weights from PyTorch's torchvision library as a starting backbone. The first convolutional layer was modified from 3-channel RGB input to accept single-channel grayscale images (Conv2d(1, 32, 3, stride=2, padding=1)). The classification head was replaced with a linear layer projecting from MobileNetV2's last channel dimension (1,280) to three output classes. A differential learning rate strategy was applied during fine-tuning: the pretrained feature extraction layers were updated at a learning rate of $1e-4$, while the new classification head was trained at $1e-3$.

5.2.3 Spiking Neural Network (SNN)

The SNN combines a convolutional feature extractor with spiking Leaky Integrate-and-Fire (LIF) classifier layers, implemented using the snnTorch library. The feature extractor consists of two convolutional blocks ($1 \rightarrow 16 \rightarrow 32$ channels, each with Batch Normalisation, ReLU, and Max Pooling), producing a $32 \times 16 \times 16$ feature map. This is flattened and passed through three fully connected spiking layers of dimensions $8,192 \rightarrow 256$, $256 \rightarrow 128$, and $128 \rightarrow 3$, each using a LIF neuron with a learnable decay parameter β initialised at 0.85 and trained with the fast sigmoid surrogate gradient function (slope=25).

The SNN operates over 25 timesteps per forward pass. At each timestep, the fixed convolutional features are passed through the spiking layers, producing binary spikes via the LIF mechanism. Spikes are accumulated across all timesteps using rate coding, the sum of spike counts is used as the classification logit.

5.3 Training Configuration

All models were trained for 10 epochs with a batch size of 32 and weight decay of $1e-4$ using the Adam optimiser. Cross-entropy loss was used for all three models. Learning rate schedules differed by architecture:

- Tiny CNN: StepLR scheduler with step size 5 and decay factor 0.5, base learning rate $1e-3$.
- MobileNetV2: CosineAnnealingLR scheduler over 10 epochs, with differential learning rates ($1e-4$ backbone, $1e-3$ head).
- SNN: OneCycleLR scheduler with maximum learning rate $5e-4$, computed over total training steps.

All experiments were conducted on Google Colab with an NVIDIA T4 GPU, simulating a resource-constrained compute environment. A fixed random seed of 42 was applied to PyTorch, NumPy, and Python random modules to ensure reproducibility across all experiments.

5.4 Evaluation Metrics

Models were evaluated on a held-out test set comprising 20% of each dataset, never seen during training. Four metrics were recorded for each experiment:

- Test Set Accuracy (%): Proportion of correctly classified images on the held-out test set.
- Parameter Count: Total number of trainable parameters, reported in millions (M).
- Inference Time (ms/image): Wall-clock time for a single batch inference divided by batch size, measured on T4 GPU after one warm-up pass.
- Energy Consumption: Estimated per-image inference energy using both methods described in Section 5.5.

5.5 Energy Measurement Methods

5.5.1 MAC-Based Theoretical Estimation

Following the methodology of Kucik and Meoni (2021), theoretical energy per inference was computed as the product of multiply-accumulate operations (MACs) and published hardware energy constants. MACs were counted using the torchinfo library. Published energy-per-MAC constants (in nanojoules) from Degnan et al. (2016) are: GPU (T4-class): 0.30 nJ/MAC; CPU (Intel i7): 8.60 nJ/MAC; ARM Cortex-A: 0.90 nJ/MAC; Loihi neuromorphic: 0.02711 nJ/MAC.

For the SNN, effective MACs were scaled by the mean spike firing rate, measured using forward hooks on the LIF layers during a single test batch. This captures the sparsity advantage of spiking computation, where only active synapses consume energy. The resulting energy estimate in microjoules (μJ) represents the expected energy cost if the model were deployed on each hardware platform.

5.5.2 CodeCarbon Real Measurement

The CodeCarbon library was used to measure actual CPU and GPU power draw during inference. For each model, 10 inference batches were run under CodeCarbon tracking, and total energy in kilowatt-hours (kWh) was recorded. Per-image energy in nanowatt-hours (nWh) and equivalent CO₂ in micrograms were computed by dividing by total images processed. Where hardware sensors were unavailable, a fallback estimate was computed using the T4 GPU thermal design power (TDP) of 70W multiplied by elapsed inference time and a 30% utilisation factor.

Chapter 6: Results

6.1 Accuracy Results

Table 5 presents the test set accuracy for each model across all four experiments. Figure 1 shows these results as a grouped bar chart. The highest single-experiment accuracy was achieved by Tiny CNN on AI4Mars with minimal preprocessing (83.3%), marginally outperforming MobileNetV2 on the same configuration (81.2%). MobileNetV2 achieved the best average accuracy across all four experiments.

Experiment	Tiny CNN	MobileNetV2	SNN
A1	72.73%	76.47%	67.38%
A2	63.10%	70.59%	45.45%
B1	83.30%	81.20%	75.60%
B2	77.20%	77.50%	62.90%

Table 5: Accuracy results (%) for all model-experiment combinations

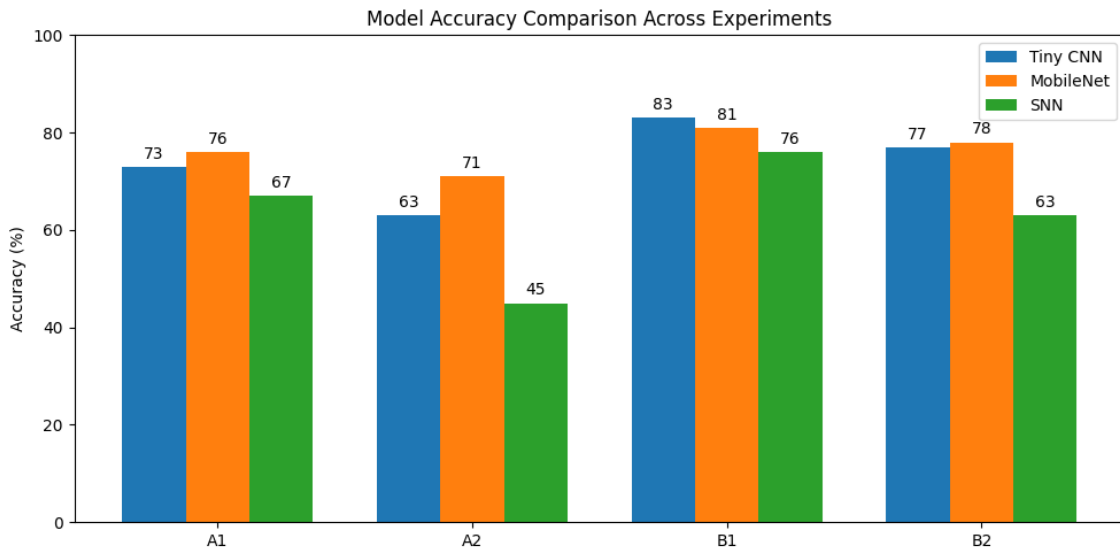


Figure 1: Accuracy

Several patterns are notable in these results. First, minimal preprocessing consistently outperforms augmented preprocessing for all models, a counterintuitive finding discussed in detail in Section 6.1. Second, Tiny CNN's 83.3% result on AI4Mars B1 exceeds MobileNetV2's performance on the same configuration, contrary to the expectation that a pretrained model should outperform a model trained from scratch. Third, the SNN's accuracy is substantially lower than both CNN-based models in every experiment, with the largest gap occurring in the augmented 1000Samples condition (A2) where the SNN achieves only 45.5%.

6.1.1 Confusion Matrix Analysis

Confusion matrices for all experiments are shown in Figures 2–5. These provide insight into class-level performance beyond overall accuracy.

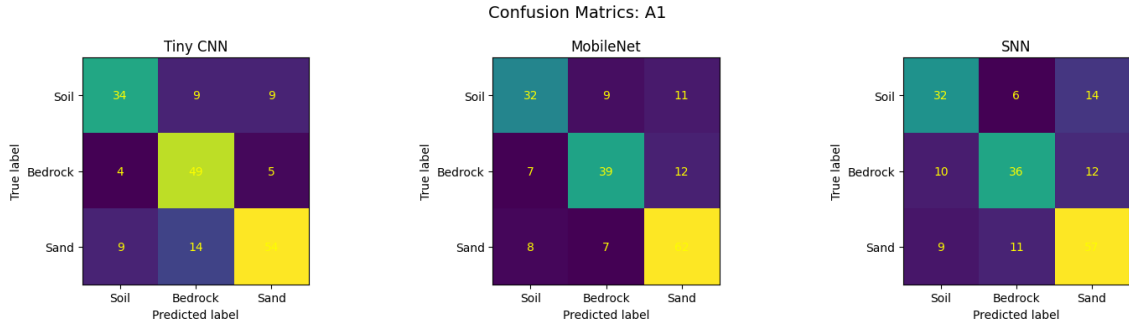


Figure 2: Confusion Matrix for Experiment A1

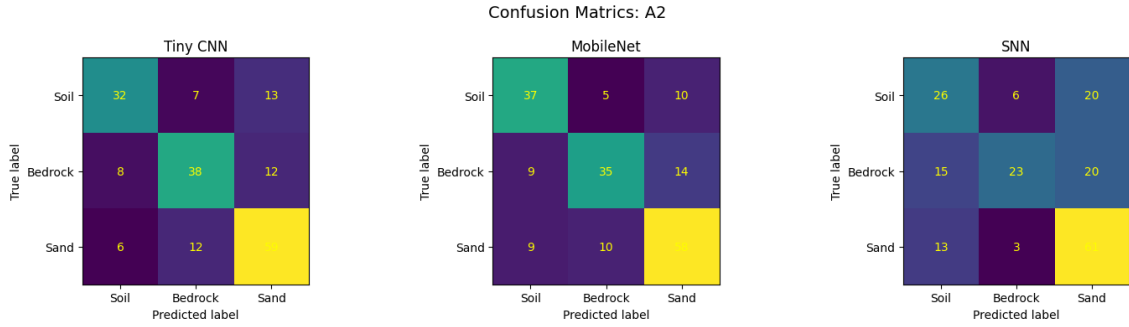


Figure 3: Confusion Matrix for Experiment A2

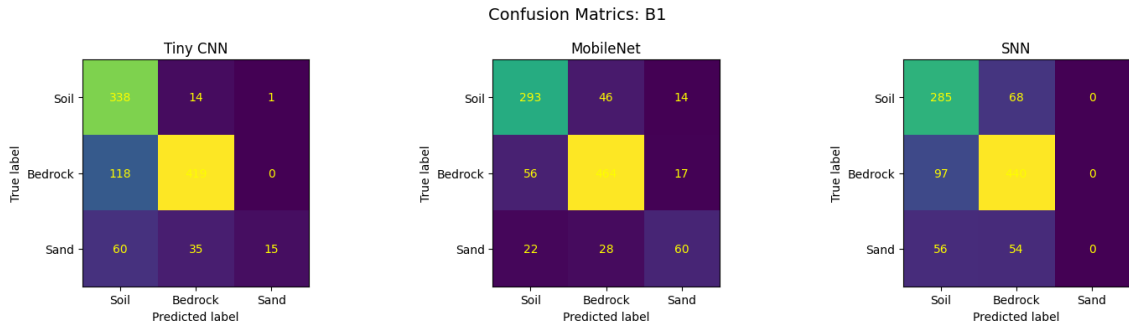


Figure 4: Confusion Matrix for Experiment B1

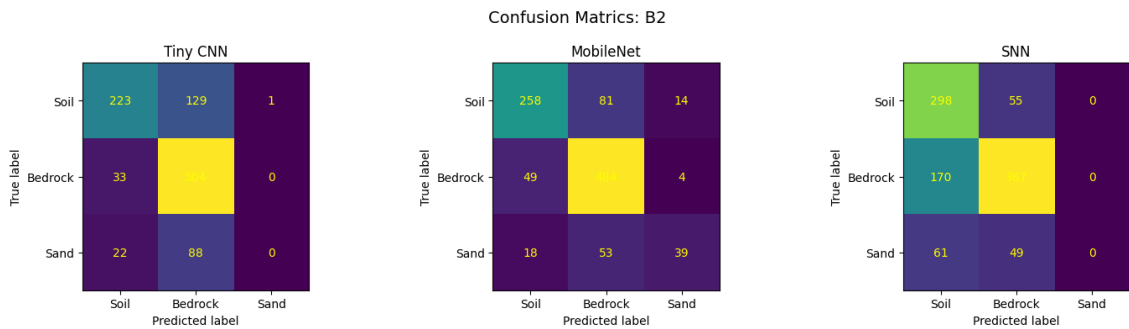


Figure 5: Confusion Matrix for Experiment B2

Across all experiments, both Tiny CNN and MobileNetV2 exhibit strong diagonal dominance, indicating effective classification across all three terrain classes. However, consistent misclassification is observed between Soil and Sand, which share similar visual characteristics in grayscale imagery.

The SNN demonstrates significantly weaker class discrimination. In the AI4Mars experiments (B1 and B2), the SNN collapses predictions toward dominant classes, with near-zero correct predictions for the Sand class. This indicates that while the SNN captures general patterns, it struggles to distinguish minority or visually similar classes.

The confusion matrices therefore reinforce the accuracy results by highlighting that the performance gap is not only in overall accuracy but also in class-level reliability, which is critical for safe terrain navigation.

6.2 Model Size and Inference Speed

Table 6 presents model size and inference speed metrics, averaged across experiments (these metrics are dataset-independent for Tiny CNN and MobileNetV2, and only marginally variable for the SNN due to differing spike rates).

Metric	Tiny CNN	MobileNetV2	SNN
Parameters	1.07M	2.23M	2.14M
Avg Inference Time	0.025 ms/img	0.190 ms/img	1.070 ms/img

Table 6: Model size and inference speed metrics

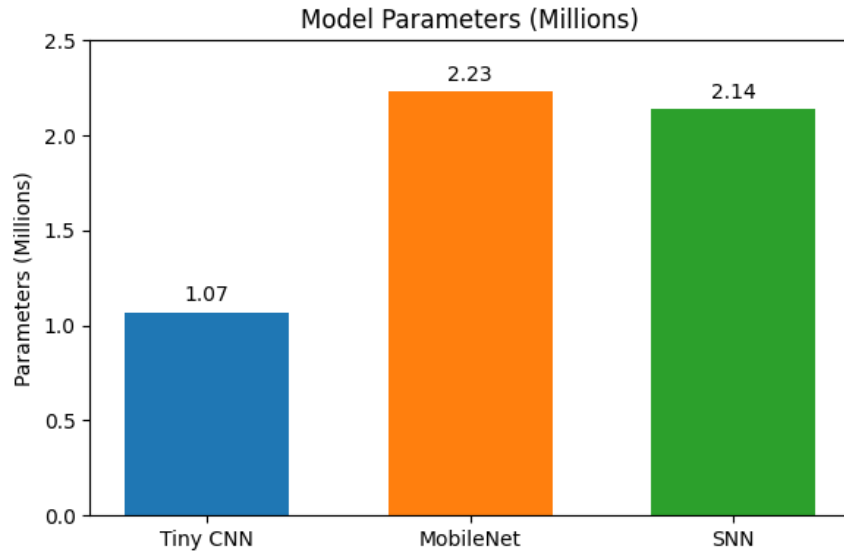


Figure 6: Model Size

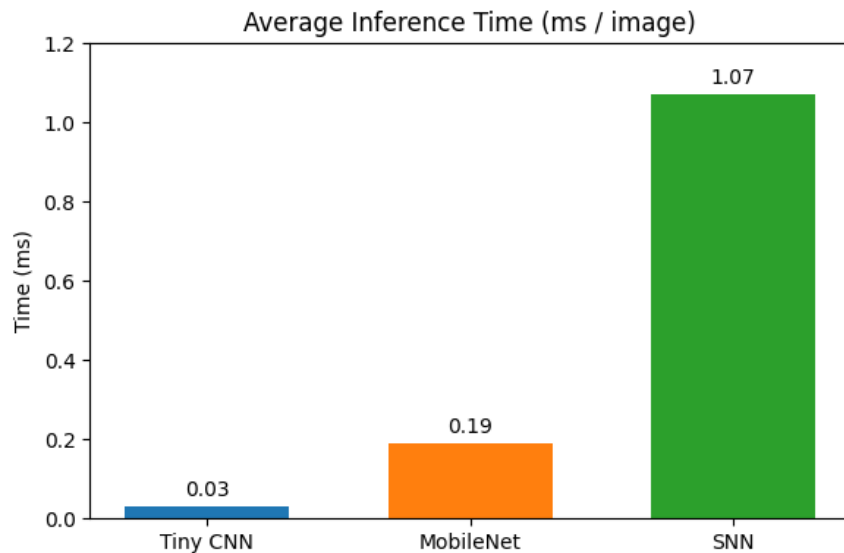


Figure 7: Inference Time

Tiny CNN is unambiguously the most efficient model: smallest parameter count and fastest inference at 0.025ms per image, approximately 7.6x faster than MobileNetV2 and 42.8x faster than the SNN. The SNN's high inference time is a consequence of sequential timestep simulation on a synchronous GPU architecture.

6.3 Energy Consumption

6.3.1 MAC-Based Theoretical Energy

Table 7 presents MACs and the theoretical energy per inference (MACs scaled by mean spike firing rate (0.20) for the SNN).

	Tiny CNN	MobileNetV2	SNN
Raw MACs	11.19M	23.90M	58.67M
Energy (T4 GPU) (μJ)	3,357	7,169	3,465

Table 7: MAC-based theoretical energy per inference (μ J)

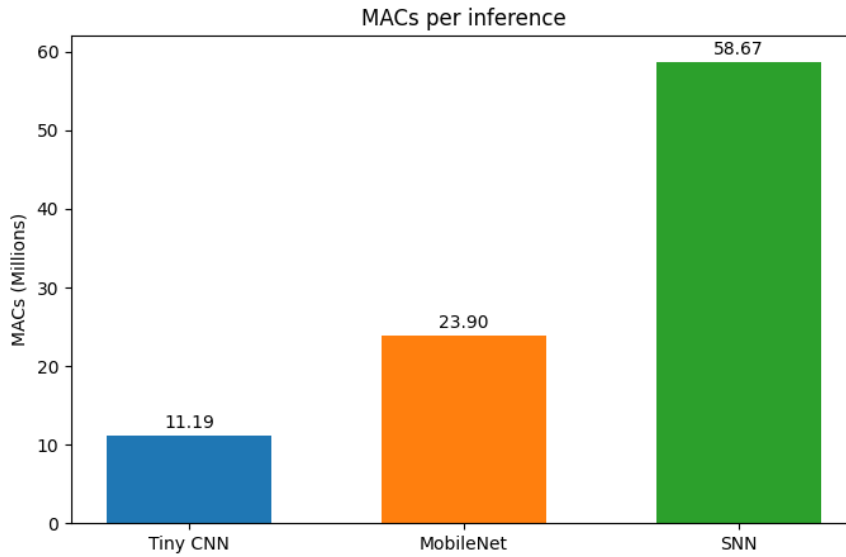


Figure 8: MACs per Inference

The SNN's 58.67M raw MACs appear to suggest high computational cost, but this figure does not account for spike sparsity.

The MAC-based analysis reveals that once spike sparsity is accounted for, the SNN's effective energy cost is comparable to Tiny CNN, and approximately 52% lower than MobileNetV2. On Intel Loihi, the most relevant platform for neuromorphic deployment, the SNN would only be 314 μ J per inference compared to 648 μ J for MobileNetV2 and 303 μ J for Tiny CNN. This result validates the theoretical energy advantage of sparse spiking computation and motivates future neuromorphic deployment.

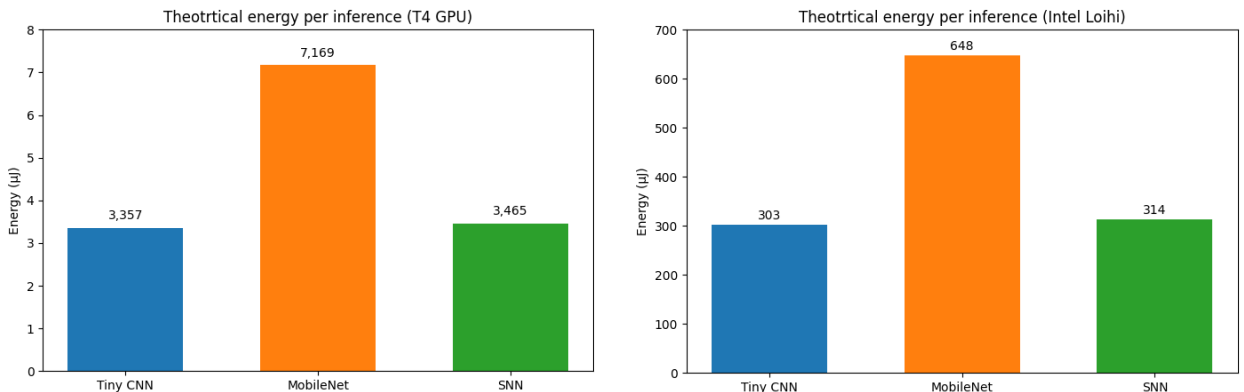


Figure 9 & 10: Theoretical energy per inference (μ J) comparison on T4 and Loihi.

6.3.2 CodeCarbon Real Measurement

Table 8 presents real measured energy from CodeCarbon, averaged across all four experiments. These values reflect actual CPU and GPU power draw on the Colab T4 GPU during inference.

	Tiny CNN	MobileNetV2	SNN
nWh / image	8.58	20.14	33.93
µg CO₂ / image	2.45	5.75	9.69

Table 8: CodeCarbon real measured energy per inference

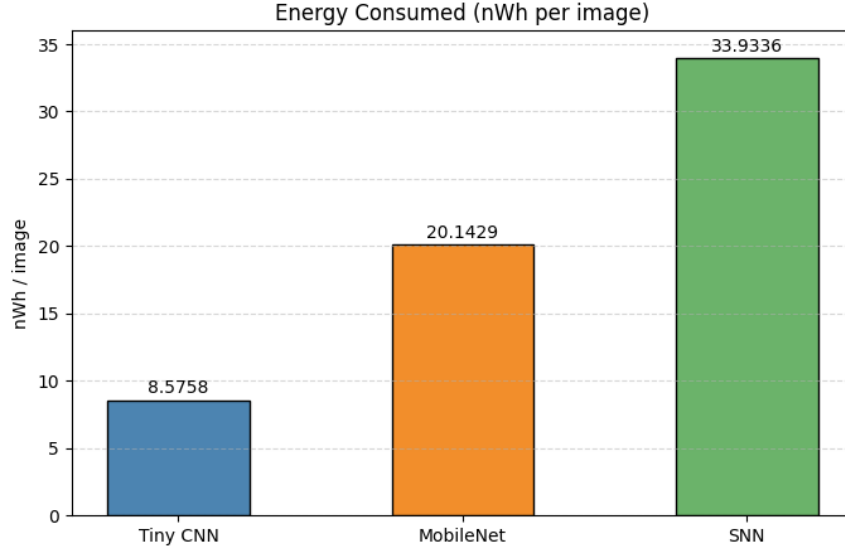


Figure 11: Real energy per inference

The CodeCarbon measurements reveal a contrast with the MAC-based estimates. On actual GPU hardware, the SNN consumes approximately 3.96× more energy per image than Tiny CNN (33.93 nWh vs 8.58 nWh), and MobileNetV2 consumes 2.35× more than Tiny CNN (20.14 nWh vs 8.58 nWh). These measurements reflect real CPU and GPU power draw during inference and do not account for spike sparsity, which is invisible to synchronous hardware. The MAC-based estimates in Table 7, by contrast, scale SNN energy by its mean spike firing rate of 0.20, capturing the theoretical sparsity advantage that would be realised on asynchronous neuromorphic hardware. The two methods are therefore answering different questions: CodeCarbon measures what actually happens on a GPU today, while the MAC-based estimate models what would happen on Loihi. Both are valid and their apparent disagreement is not a methodological error, it reflects the fundamental hardware dependency of neuromorphic efficiency, discussed in detail in Section 7.3.

Chapter 7: Discussion

7.2 Impact of Data Augmentation

Across all three models and both datasets, augmented preprocessing (A2, B2) consistently produced lower test accuracy than minimal preprocessing (A1, B1). The accuracy drops are substantial in some cases: the SNN on 1000Samples drops from 67.4% (A1) to 45.5% (A2), and Tiny CNN on 1000Samples drops from 72.7% to 63.1%.

This finding is counterintuitive given that data augmentation is widely understood to improve model generalisation. However, the explanation lies in the interaction between augmentation intensity and training duration. Data augmentation is fundamentally a regularisation technique, it artificially expands the training distribution by introducing variance, which forces the model to learn more robust features. This benefit materialises over many training epochs as the model encounters sufficient repetitions of each augmented variant to learn the underlying invariances.

With only 10 training epochs, models did not have sufficient exposure to the augmented distribution to converge effectively. The aggressive transformations (rotations of ± 15 -20 degrees, colour jitter of ± 0.3 -0.5) introduced enough variance that models were unable to fit the training data adequately in the available time, resulting in under-fitting relative to the minimal pipeline. This interpretation is consistent with the observation that training losses on augmented experiments were consistently higher than minimal experiments at epoch 10, suggesting the models had not yet converged. Training for 50 or more epochs would be expected to reverse this trend.

7.3 Tiny CNN vs MobileNetV2

The result that Tiny CNN outperforms MobileNetV2 on AI4Mars B1 (83.3% vs 81.2%) warrants careful analysis. On the smaller 1000Samples dataset, MobileNetV2 consistently outperforms Tiny CNN across both preprocessing conditions, which is the expected pattern for a pretrained model with a large feature-learning head start.

On the larger, more diverse AI4Mars dataset, Tiny CNN's advantage can be attributed to domain mismatch in the MobileNetV2 pretrained weights. MobileNetV2 was pretrained on ImageNet — a dataset of natural, high-resolution, three-channel colour images of everyday objects. The features learned by MobileNetV2's early layers are tuned for colour gradients, high-frequency textures, and object boundaries of a fundamentally different character than the low-contrast, monochromatic rock and dust textures of Martian terrain.

Tiny CNN, by contrast, learns its features entirely from the Mars training data. On the smaller 1000Samples dataset, Tiny CNN lacks sufficient data to learn competitive features from scratch, giving MobileNetV2's pretrained initialisation a decisive advantage. On the larger AI4Mars dataset with 4,000 training images, Tiny CNN has enough data to learn domain-appropriate features that outperform the transferred — and potentially mismatched — ImageNet features. This suggests that for Mars-specific deployment with adequate training data, lightweight domain-specific models may be preferable to pretrained general-purpose architectures.

7.4 SNN Performance and Hardware Dependency

The SNN consistently underperformed compared to both CNN-based models in classification accuracy and consumed dramatically more energy on Colab's T4 GPU according to CodeCarbon measurements (33.79 nWh/image vs 8.58 nWh for Tiny CNN). These results appear to contradict the theoretical energy advantage demonstrated in the MAC-based analysis (Table 7).

This apparent contradiction is resolved by understanding the fundamental difference between synchronous and asynchronous computing. The T4 GPU is a synchronous processor: its clock ticks at a fixed rate and all circuits perform operations regardless of whether meaningful computation is occurring. Simulating 25 SNN timesteps sequentially on such hardware requires 25 forward passes through the spiking layers per inference, regardless of spike sparsity. The GPU cannot exploit the fact that 80% of neurons are silent, every neuron is evaluated at every timestep regardless.

Neuromorphic hardware such as Intel Loihi operates asynchronously, computation is event-driven, occurring only when a spike signal arrives at a neuron. On such hardware, a neuron with a 0.20 mean firing rate literally performs only 20% of the operations of a fully active neuron, directly translating sparsity to energy savings. This is precisely the source of the 52% energy reduction shown in Table 7. As Kucik and Meoni (2021) state: the energy advantages of SNNs are hardware-dependent and best realised on dedicated neuromorphic platforms.

The practical implication is that the SNN, as implemented, is not the optimal choice for conventional GPU or CPU deployment. However, as neuromorphic processors become increasingly viable for embedded and space applications. Intel has already demonstrated Loihi in robotic systems, and neuromorphic hardware is being investigated for ESA satellite applications, the SNN architecture becomes significantly more attractive. The consistently sparse firing rates observed (0.19-0.22 across all experiments) confirm that the SNN is functioning as designed and would realise substantial energy savings on appropriate hardware.

7.5 Energy Consumption Consideration

A broader perspective on energy consumption is relevant to the Mars rover deployment context. The terrain classification inference energies measured in this project, ranging from 303 μJ (Tiny CNN on Loihi) to 205,511 μJ (MobileNetV2 on CPU), are extremely small relative to the total power budget of a rover. Curiosity's total power consumption is approximately 110W; even if terrain classification runs continuously at 10 frames per second, the inference energy represents a negligible fraction of total rover power consumption.

This observation has an important implication: explicitly optimising the loss function for energy consumption is not warranted in this context. Energy efficiency matters for terrain classification primarily as a proxy for computational feasibility, ensuring the model can run in real time on the available processor, rather than as a direct contribution to the rover's energy budget. The more relevant constraints are inference time (must complete within the rover's control cycle), model size (must fit within available RAM), and accuracy (must be sufficient for safe navigation decisions).

Chapter 8: Conclusion

8.1 Conclusions

This project presented a systematic comparative evaluation of three lightweight neural network architectures for Martian terrain classification, motivated by the real constraints of planetary rover deployment: severe compute limits, strict power budgets, and the impossibility of real-time remote intervention due to Earth-Mars communication delays.

The key findings are as follows. Tiny CNN achieves the best balance of accuracy, speed, and efficiency for rover deployment, reaching 83.3% test accuracy on the AI4Mars dataset with minimal preprocessing while requiring only 1.07M parameters and 0.025ms per inference. It is the most viable architecture for deployment on current rover hardware. MobileNetV2 achieves slightly higher average accuracy across experiments and is a viable alternative when memory permits, but its pretrained ImageNet features do not transfer as effectively to the Mars domain as expected, particularly on the larger and more diverse AI4Mars dataset.

The SNN presents a complex picture. On conventional GPU hardware it underperforms both accuracy and energy metrics, consuming approximately 4 times more energy than Tiny CNN during inference. However, its consistently sparse spike firing rates (0.19-0.22) indicate the architecture is functioning as designed, and theoretical MAC-based estimates show a 52% energy advantage over MobileNetV2 on Intel Loihi neuromorphic hardware. As neuromorphic processors move toward space-grade deployment, the SNN architecture becomes increasingly competitive.

Data augmentation consistently reduced accuracy under the 10-epoch training schedule used, confirming that augmentation is a regularisation tool that requires sufficient training duration to yield its generalisation benefits. This is an important practical consideration for training AI models under compute-constrained conditions.

Together these findings provide a practical roadmap for lightweight AI deployment on planetary rovers. The appropriate choice of architecture depends critically on the target hardware: Tiny CNN for conventional embedded processors today, and SNN for neuromorphic hardware in future missions. As Andrzej S. Kucik and Gabriele Meoni of the European Space Agency observed, one of the primary factors limiting the deployment of AI models in space is the restricted energy budget, and this project demonstrates that addressing that constraint requires both careful architectural choice and hardware-algorithm co-design.

8.2 Limitations

- Training was capped at 10 epochs and 64×64 grayscale resolution due to Colab compute limits, both well below the scales used in published Mars terrain classification research.
- AI4MARS was sampled at 5,000 images (of 16,064 available) and labels were simplified from pixel-level masks to image-level majority vote, losing spatial terrain detail.
- The SNN was benchmarked on a conventional GPU, an architecturally mismatched platform that cannot exploit spike sparsity, making its energy and speed results an upper bound rather than a fair evaluation.
- Models were not evaluated on any embedded or rover-class hardware; all inference times are GPU estimates.
- The task was simplified from semantic segmentation to image-level classification, which removes spatial context and limits applicability to real-world navigation.

8.3 Future Work

- Train for 50+ epochs at higher resolution (128×128 or 256×256) to fully evaluate augmentation's generalisation benefit and close the accuracy gap with published benchmarks.
- Deploy the SNN on Intel Loihi or SpiNNaker to obtain real energy measurements and validate the 52% theoretical energy saving over MobileNetV2.
- Apply INT8 post-training quantisation to Tiny CNN to reduce its footprint below 1MB, potentially making it viable for Zhurong's 20MB RAM constraint.
- Extend from image-level classification to pixel-level semantic segmentation using a lightweight decoder, enabling precise hazard localisation for path planning.
- Benchmark inference time and power draw on embedded hardware (e.g., NVIDIA Jetson Nano or Raspberry Pi) to validate real-world deployment feasibility.
- Investigate SNN sparsity regularisation during training to push mean spike rate below 0.15, amplifying energy savings on neuromorphic hardware without accuracy loss.

References

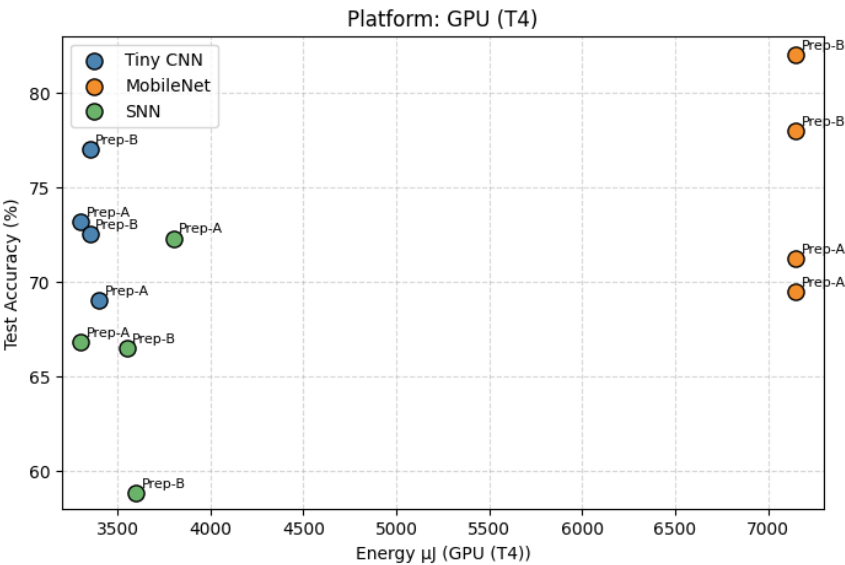
- Feng, L., Wang, S., Wang, D., Xiong, P., Xie, J., Hu, Y., Zhang, M., Wu, E. Q., & Song, A. (2025). Mobile-DeepRFB: A lightweight terrain classifier for automatic Mars rover navigation. *IEEE Transactions on Automation Science and Engineering*, 22, 17442–17451. <https://doi.org/10.1109/TASE.2023.3340190>
- Ibrahim, Z. I., & Al-Jamali, N. A. S. (2024). Single spike neural network model for superficial environment classification for mobile robot navigation. *Journal of Engineering*, 30(4), 118–133. <https://doi.org/10.31026/j.eng.2024.04.08>
- Kucik, A. S., & Meoni, G. (2021). Investigating spiking neural networks for energy-efficient on-board AI applications: A case study in land cover and land use classification. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 1–10.
- Lin, B., Wang, F., Li, Q., Zheng, B., Yao, M., Xiao, X., Qi, Y., Cui, H., & Huang, X. (2025). LisseMars: A lightweight semantic segmentation model for Mars helicopter. *Aerospace*, 12(12), 1049. <https://doi.org/10.3390/aerospace12121049>
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 4510–4520.
- Swan, R. M., Atha, D., Leopold, H. A., Gildner, M., Oij, S., Chiu, C., & Ono, M. (2021). AI4MARS: A dataset for terrain-aware autonomous driving on Mars. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 1982–1991.
- Xing, Y., Wei, C. L., & Tang, L. (2021). Development of autonomous sensing and control technology for extraterrestrial mobile exploration unmanned systems. *Aerospace Control and Application*, 47(6), 1.

Appendix:

The following figures provide additional analysis supporting the results presented in Chapter 6. These plots illustrate the relationship between accuracy, energy consumption, and preprocessing strategies across different models and datasets.

While not essential for the primary results, they offer further insight into model behavior, particularly in terms of robustness, efficiency trade-offs, and the impact of data augmentation.

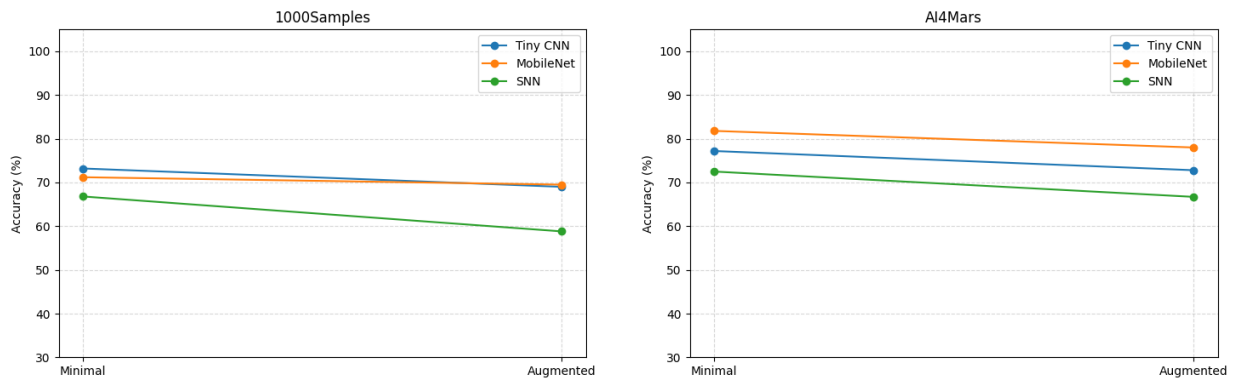
A.1 Energy–Accuracy Trade-off



This figure shows the relationship between energy consumption and classification accuracy for all models when executed on a GPU (T4). Each point corresponds to a specific preprocessing configuration.

The results highlight a clear trade-off between performance and energy usage. MobileNet achieves the highest accuracy but at the cost of significantly higher energy consumption. TinyCNN operates in a lower-energy regime while maintaining competitive accuracy, indicating a strong efficiency advantage. The SNN exhibits moderate energy usage but lower accuracy, reinforcing the observation that its benefits are not realized in conventional hardware environments.

A.2 Effect of Augmentation



These plots illustrate the impact of data augmentation on model accuracy across both datasets. All models experience a reduction in accuracy when augmentation is applied, indicating increased task difficulty due to added variability. The SNN shows the largest performance drop, suggesting limited robustness to distribution shifts. In contrast, Tiny CNN and MobileNet demonstrate more stable behavior, although a decrease in performance is still observed.

A similar trend is observed for Dataset B (AI4Mars), where augmentation also reduces accuracy across all models. However, the performance degradation is less severe compared to Dataset A, suggesting that models trained on larger datasets are better able to generalize under increased variability. MobileNet remains the most robust, while the SNN continues to exhibit the greatest sensitivity to augmentation.